

SDP



GitHub Copilot Hooks

Programmable Governance for Agent Workflows

Real-time control and compliance at every agent lifecycle moment

"How do you govern AI agents without destroying their velocity?"

Agents create files, run commands, and deploy changes autonomously. Manual gates destroy velocity, but

hooks give you synchronous enforcement at every lifecycle moment.

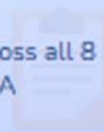
Security Architects

Real-time denial of dangerous operations in
<2 seconds — before execution



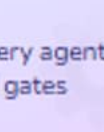
Compliance Officers

Complete JSON Lines audit trail across all 8
lifecycle events for SOC2 and HIPAA



DevOps Engineers

Programmatic governance at every agent
action without manual approval gates



8 lifecycle events

Complete coverage from SessionStart to Stop
— no action escapes the governance scaffold

PreToolUse only

The only hook that can deny execution before
it happens — all others observe after

<2 seconds

Synchronous hook execution time —
governance without velocity loss or timeout
risk

PART 1

Phase 1: Lifecycle Control

Eight moments, one mental model

Map the events from SessionStart to Stop — build the governance scaffold

PART 2

Phase 2: PreToolUse

The only hook that prevents execution

Deny dangerous operations in <2 seconds before damage occurs

PART 3

Phase 3: Audit Trail

JSON Lines logging and jq compliance

Complete lifecycle coverage — 100% audit trail with zero sampling

PART 4

Phase 4: Real-World Patterns

HIPAA, SOC2, and quality gate configs

Copy-paste patterns deployable within hours of this session

Click any section to jump directly there

Phase 1: Lifecycle Control

Eight lifecycle moments from SessionStart to Stop — build the mental model before exploring the hooks



8 Lifecycle Events

Complete SessionStart to Stop
governance scaffold



JSON Configuration

Define hooks in `.github/hooks/*.json`
files



Exit Code Semantics

0 = success · 2 = blocking · other =
warning

Hooks run synchronously in the agent execution path
↳ Every action – every moment – every session

Eight Lifecycle Events — Part 1: Session and Tool

Four events cover session initialization, user prompts, and tool execution

SessionStart	Initialize resources and inject project context into the session	Context
PromptSubmit	Audit user requests and inject system context into prompts	Audit
PreToolUse	Deny dangerous ops, require approval, or modify tool input	⚡ Prevents
PostToolUse	Run formatters, log results, enforce quality standards	Observe

PreToolUse is the only event that can prevent — all others observe, enrich, or control flow

Eight Lifecycle Events — Part 2: Compaction and Subagents

Four events cover context preservation, nested agents, and session finalization

PreCompact	Export context before truncation and save session state	Preserve
SubagentStart	Track nested agents and inject guidelines for subagents	Spawn
SubagentStop	Aggregate results and verify subagent output quality	Aggregate
Stop	Generate reports, cleanup, or enforce completion requirements	Finalize

Stop and SubagentStop can block completion — direct agents to finish required work before ending

Hooks Are Defined in JSON — No Framework Required

.github/hooks/security-hooks.json

```
{
  "hooks": {
    "PreToolUse": [{
      "type": "command",
      "command": "./scripts/security-check.sh",
      "cwd": ".github/hooks",
      "timeout": 5
    }],
    "PostToolUse": [{
      "type": "command",
      "command": "./scripts/format-changed.sh"
    }],
    "SessionStart": [{
      "type": "command",
      "command": "./scripts/log-session.sh"
    }]
  }
}
```

type: command

Spawns a shell process — bash, PowerShell, or any executable

OS-specific overrides

windows / linux / osx keys fall back to command if absent

timeout: default 30s

Target under 5 seconds — hooks run in the agent path

Phase 2: PreToolUse Enforcement

PreToolUse is the only hook that can deny execution before it happens — every other approach is post-incident



Only Preventive Hook

PreToolUse is the only hook with veto power



Three Decisions

deny · ask · allow — matched to risk level



Modify Tool Input

updatedInput enforces safe defaults before exec

deny, ask, or allow – before the tool runs anything
↳ PreToolUse: the only hook that prevents

PreToolUse Is the Only Hook That Prevents

Every other governance approach is post-incident

<2s

synchronous execution time — governance runs
before the tool, not after the damage

Target: <2s for security checks; <5s for linting and validation

🚫 **deny** — hard block

Stops execution immediately before the tool runs

💬 **ask** — soft gate

Surfaces to user for approval — model pauses and waits

✏️ **updatedInput** — modify

Adjust tool args before allowing (e.g., add --dry-run)

💡 **additionalContext** — enrich

Inject guidance without blocking the operation

🔄 All other hooks observe after the fact. Only PreToolUse can prevent.

What Your Script Returns — Three Decision Shapes

stdout — PreToolUse hook response

```
{
  "hookSpecificOutput": {
    "hookEventName": "PreToolUse",
    "permissionDecision": "deny",
    "permissionDecisionReason": "rm -rf outside approved dir"
  }
}

// OR — modify before allowing:
{
  "hookSpecificOutput": {
    "hookEventName": "PreToolUse",
    "permissionDecision": "allow",
    "updatedInput": { "command": "rm -rf dist --dry-run" }
  }
}
```

permissionDecisionReason

Shown to the model — it uses this to explain and self-correct

updatedInput

Replaces tool args — must match the tool schema exactly

First deny wins

Multiple hooks chain in order — most restrictive takes precedence

Security Denial in Action

From dangerous command intent to blocked — before execution

```
$ copilot
> "Remove old build artifacts from the dist directory to free up space"
🤖 Agent:
  I will use the terminal to clean up the directory.
  Running: rm -rf dist/old-artifacts

🔒 PreToolUse Hook (1.8s):
  Pattern match: rm -rf in unapproved path
  Decision: DENY — Destructive path operation blocked by security policy

✅ Operation blocked before execution — zero damage
✅ Agent: I cannot complete this — please specify individual files to remove
```

1.8 seconds — from rm command intent to denied, before execution

Three Decisions — Map Each to a Risk Level

deny + ask: Control Decisions

deny — hard block

Dangerous commands, privilege escalation, database destruction

ask — soft gate

Production writes, data exports, irreversible operations

First deny in a chain wins — most restrictive takes precedence

permissionDecisionReason guides the model to self-correct

allow + updatedInput: Safe Defaults

allow — explicit approval

Reduce false positives for confirmed safe operations

updatedInput — modify args

Add --dry-run, restrict paths, enforce safe flags

additionalContext injects project guidance without blocking

Verify updatedInput schema via the Chat Debug View

Phase 3: Observability & Audit

JSON Lines logging across all eight events enables direct jq queries and SIEM integration for SOC2 and HIPAA



JSON Lines Format

Append-only .jsonl — concurrent-safe
and jq ready



All 8 Events

Complete lifecycle — zero gaps in the
audit trail



SIEM Integration

Direct import to Datadog, Splunk,
Elasticsearch

```
jq select .permissionDecision=deny logs/audit.jsonl  
↳ Auditors query directly – no manual sampling
```


One Line Per Event — Direct jq Queries, No Parsing

logs/audit.jsonl

```
{"timestamp": "2026-02-06T17:30:00Z", "event": "SessionStart", "to"}
{"timestamp": "2026-02-06T17:30:15Z", "event": "PreToolUse", "to"}
{"timestamp": "2026-02-06T17:30:25Z", "event": "PostToolUse", "t"}
{"timestamp": "2026-02-06T17:30:30Z", "event": "Stop", "toolsUse"}
```

jq queryable

Find all denials: `jq select .decision==deny` — no grep needed

Append-safe

Concurrent writes do not corrupt — one JSON object per line

SIEM compatible

Datadog, Splunk, and Elasticsearch all ingest JSONL natively

Four Reasons Compliance Teams Need This Audit Trail



100% Coverage

Every agent action across all 8 lifecycle events — not sampling, not spot checks



Direct Query

jq select by tool, decision, or timestamp — no custom log parser needed



Append-Only

Immutable per-line append — concurrent writes safe, no corruption risk



SIEM Ready

Import to Splunk, Datadog, or Elasticsearch for alerting and dashboards



SOC2 and HIPAA auditors get enforcement evidence — not sampling. Audit prep shrinks from weeks to hours.

Two More Hooks That Pay Dividends

SessionStart — Context Injection

additionalContext in output

Injected into the agent conversation at start

Project-specific metadata

Branch, version, environment, team standards

SubagentStart works the same way for nested agents

Eliminates manual agent setup for every session

PostToolUse — Quality Gates

Fires after every tool completes

editFiles, createFile, runTerminalCommand

Run prettier and ESLint

Standards enforced at creation not in CI

additionalContext tells agent about lint errors to fix

decision: block stops further processing if quality fails

Phase 4: Real-World Patterns

HIPAA, SOC2, and multi-layer security patterns — copy-paste configurations
deployable within hours



Security Patterns

Multi-layer PreToolUse for defense-in-depth



Quality Gates

PostToolUse runs prettier and lint after edits



Context Injection

SessionStart fills agents with project context

HIPAA, SOC2, quality, and context – four patterns you can deploy today
↳ 1-2 hours from zero to governed agent workflows

Three Regulatory Patterns — Each Deployable in Hours



HIPAA

Healthcare PHI protection

Deny access to PHI paths outside approved tools

Log all 8 events — retention required by regulation

ask for any data export operation

SessionStart validates project authorization state



SOC2

Security and availability

Multi-layer PreToolUse: commands + files + secrets

JSON Lines to SIEM — direct audit evidence

PreCompact exports context before truncation

Stop hook generates end-of-session compliance report



Code Quality

Shift-left enforcement

PostToolUse: prettier + ESLint after every editFiles

Lint errors injected as additionalContext to fix now

Stop hook blocks session until test suite passes

SessionStart injects team style guide and standards

Four Patterns — Combine for Defense-in-Depth

Each pattern solves one class of governance problem — layer them for complete coverage

Multi-Layer

Multiple PreToolUse hooks run in order; first deny wins

Defense depth

Quality Gate

PostToolUse runs prettier + ESLint after every editFiles call

Shift left

Env Policies

Production=deny · staging=ask · development=allow by CWD

Environment

Ctx Injection

SessionStart + SubagentStart inject branch and project info

Context

Start with Multi-Layer security. Add Quality Gate in week 2. Env Policies and Context Injection in month 1.

Manual Governance vs Copilot Hooks

The same agent action — two very different outcomes

✗ Manual Governance

- 1 Request submitted
Agent begins autonomous work
- 2 Agent takes action
No enforcement gate — anything allowed
- 3 Violation executes
`rm -rf` or `DROP TABLE` — damage done
- 4 Post-incident review
Hours of log forensics after damage



Violation discovered after damage — hours of recovery

✓ With Copilot Hooks

- 1 Request submitted
Agent begins autonomous work
- 2 PreToolUse fires
Security policy checked in <2 seconds
- 3 Deny decision returned
Operation blocked before execution
- 4 Audit log appended
JSON Lines entry — queryable with `jq`



Blocked in <2 seconds — complete audit trail

Real-time prevention vs. post-incident recovery

From Post-Incident Review to Real-Time Prevention

Before

AI agents operate with no enforcement gates — violations found after damage

Policy violations reconstructed through hours of post-incident log forensics

CI catches quality violations after code is committed — rework cycles

Manual approval gates required for sensitive operations — velocity destroyed

After

PreToolUse denies dangerous operations in <2 seconds before execution

JSON Lines audit trail covers all 8 lifecycle events — 100% coverage

PostToolUse enforces formatting and linting at the point of creation

Synchronous governance runs in the agent path — no manual gates needed

<2s

PreToolUse execution time — real-time prevention

8 events

complete lifecycle coverage with zero gaps

100%

audit coverage — no sampling, every action recorded

✓ What You Can Do Today

Today

- ❑ Create `.github/hooks/` and deploy the security-check script to one repository
- ❑ Add `SessionStart` and `Stop` hooks to start logging all agent sessions to `audit.jsonl`
- ❑ Trigger a sandbox denial — run an agent and verify the deny response works

This Week

- ❑ Wire all 8 lifecycle hooks for a complete JSON Lines audit trail
- ❑ Add `PostToolUse` quality gates — prettier and ESLint enforcement after file edits
- ❑ Implement environment-aware policies: `production=deny`, `staging=ask`, `development=allow`

This Month

- ❑ Integrate JSON Lines logs into your SIEM for SOC2 or HIPAA compliance evidence
- ❑ Roll out multi-layer security enforcement to all production repositories
- ❑ Document hook patterns as org standards and share with your compliance team

🔑 Key Takeaway

Hooks move governance from post-incident review to real-time prevention.

 Official Documentation[Agent hooks configuration in VS Code](#)

Complete configuration reference with input/output formats

[About Copilot hooks](#)

Core concepts and hook types overview

[Hooks configuration reference](#)

Complete spec with all input and output formats

[Using hooks with coding agent](#)

Step-by-step implementation and deployment guide

 Related Content[Terminal Sandboxing](#)

OS-level execution controls — complements hooks for defense-in-depth

[Copilot Primitives](#)

Custom instructions, agents, and skills for broader customization



GitHub Copilot Hooks

Programmable Governance for Agent Workflows

PreToolUse

The only hook that prevents execution before it happens — not post-incident

8 events

Complete lifecycle coverage from SessionStart to Stop — zero gaps

<2 seconds

Synchronous governance in the agent path — prevention without velocity loss

What would your team enforce first if every agent action had a real-time gate?